

Lab 2 — Task 9: Record of Tools, Files, Commands and Observations

Akash Singh Sagar | Repository: github.com/akashsinghsagar/devops-lab2

Objective

Keep one consolidated record of everything used in the Lab 2 experiments: the Dockerfile, the Kubernetes manifests, the CI/CD workflow, the commands run for each experiment, the screenshots taken and the observations made.

1. Repository layout

```
devops-lab2/
+-- app/                # Flask app: app.py, requirements.txt, Dockerfile
+-- compose-app/        # Docker Compose setup (Task 2)
+-- storage-demo/       # bind-mount data used in the storage experiment (Task 3)
+-- k8s/
|   +-- deployment.yaml # Task 5, 6
|   +-- service.yaml    # Task 5
+-- .github/workflows/
|   +-- cd.yml          # Task 7
+-- docs/               # one document per task (Lab-2-Task-N)
+-- screenshots/        # evidence images referenced by the documents
+-- scripts/
```

2. Dockerfile

app/Dockerfile — builds the image lab2-python-app:1.0 for the Flask application (port 5000).

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]

docker build -t lab2-python-app:1.0 ./app
docker run -d --name lab2-app -p 5000:5000 lab2-python-app:1.0
```

3. Kubernetes manifests

k8s/deployment.yaml — 3 replicas, RollingUpdate strategy, health probes and resource limits.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab2-app
  labels:
    app: lab2-app
spec:
  replicas: 3
```

```

selector:
  matchLabels:
    app: lab2-app
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
template:
  metadata:
    labels:
      app: lab2-app
  spec:
    containers:
      - name: lab2-app
        image: lab2-python-app:1.0
        imagePullPolicy: IfNotPresent
        ports:
          - containerPort: 5000
        readinessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 3
          periodSeconds: 5
        livenessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 10
          periodSeconds: 10
        resources:
          requests:
            cpu: 50m
            memory: 64Mi
          limits:
            cpu: 250m
            memory: 128Mi

```

k8s/service.yaml — NodePort Service in front of the pods.

```

apiVersion: v1
kind: Service
metadata:
  name: lab2-app-svc
  labels:
    app: lab2-app
spec:
  type: NodePort
  selector:
    app: lab2-app
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080

```

4. CI/CD workflow

.github/workflows/cd.yml — builds the image, updates the manifest with the commit-SHA tag, applies k8s/ and verifies the rollout.

```
name: CD - Deploy to Kubernetes

on:
  push:
    branches: [main]
    paths: ["app/**", "k8s/**", ".github/workflows/cd.yml"]
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Start Minikube
        uses: medyagh/setup-minikube@latest
        with:
          driver: docker

      - name: Build image inside Minikube
        run: |
          echo "TAG=${GITHUB_SHA::7}" >> $GITHUB_ENV
          minikube image build -t lab2-python-app:${GITHUB_SHA::7} ./app

      - name: Point manifest at the new image
        run: |
          sed -i "s|\\(image: .*:\\).*|\\1${TAG}|" k8s/deployment.yaml
          grep image: k8s/deployment.yaml

      - name: Apply manifests
        run: kubectl apply -f k8s/

      - name: Wait for rollout
        run: |
          kubectl rollout status deployment/lab2-app --timeout=180s
          kubectl get deployments,pods,svc

      - name: Smoke test
        run: |
          URL=$(minikube service lab2-app-svc --url)
          curl -sf $URL/health
```

5. Commands used in each experiment

Task	Experiment	Commands
1	Containerize the app	<pre>docker build -t lab2-python-app:1.0 ./app docker run -d --name lab2-app -p 5000:5000 lab2-python-app:1.0 docker history lab2-python-app:1.0 docker logs lab2-app</pre>
2	Manage containers and images	<pre>docker ps -a docker stop lab2-app·docker start lab2-app docker rm lab2-app docker tag lab2-python-app:1.0 <user>/lab2-python-app:1.0 docker push <user>/lab2-python-app:1.0 docker compose up -d docker compose down</pre>
3	Docker storage	<pre>docker rm -f <container> (container-layer data is lost) docker volume create lab2-vol docker run -d -v lab2-vol:/data lab2-python-app:1.0 docker run -d -v \${PWD}/storage-demo/bind-data:/data lab2-python-app:1.0</pre>
4	Docker network	<pre>docker network create lab2-net docker run -d --name lab2-net-app --network lab2-net lab2-python-app:1.0 docker run -d --name lab2-net-client --network lab2-net curlimages/curl:latest sleep 3600 docker exec lab2-net-client curl -s http://lab2-net-app:5000/ docker exec lab2-net-client curl -s http://172.20.0.2:5000/</pre>
5	Deploy to Kubernetes	<pre>minikube start --driver=docker minikube image load lab2-python-app:1.0 kubectl apply -f k8s/ kubectl get pods -o wide kubectl get svc minikube service lab2-app-svc --url</pre>
6	Rolling update, rollback, scaling	<pre>kubectl set image deployment/lab2-app lab2-app=lab2-python-app:2.0 kubectl rollout status history undo deployment/lab2-app kubectl scale deployment lab2-app --replicas=5</pre>
7	Continuous deployment	<pre>git add,git commit,git push origin main Pipeline runs: minikube image build,kubectl apply -f k8s/, kubectl rollout status</pre>
8	Report	<pre>docker --version minikube version kubectl version --client kubectl get deployments,pods,svc</pre>

6. Screenshot index

Task	Document	Evidence shown
1–3	Lab-2-Task-1 ... Task-3	Image build, layers and container behavior; container management, tagging, push and Compose; storage experiments.
4	Lab-2-Task-4	lab2-net created; access by container name; access by IP 172.20.0.2.
5	Lab-2-Task-5	Cluster running; image loaded; manifests applied; pods and service; access through the Service.
6	Lab-2-Task-6	Strategy; rolling update; failed update; rollback; scaling; traffic distribution.
7	Lab-2-Task-7	Workflow run; job steps; apply and rollout logs; automatic redeploy.
8	Lab-2-Task-8	Tool versions; architecture diagrams; running state; revision history; scaling; CD run.

7. Observations per experiment

Task	Observation
1	An image is built from layers defined by the Dockerfile; the container runs the app on port 5000.
2	Containers can be listed, stopped, started and removed independently of the image; tagged images can be pushed to a registry; Compose starts a multi-container setup from one file.
3	Data in the container layer is lost when the container is deleted; a named volume and a bind mount keep the data.
4	On a user-defined bridge network the app is reachable by container name (embedded DNS) and by IP, with no ports published to the host.
5	A Deployment keeps three pods running and a NodePort Service exposes them; readiness probes on /health control when a pod receives traffic.
6	Rolling updates replace pods gradually without downtime; a failed update was contained and rolled back with one command; scaling changed the pod count in seconds.
7	A push to main deploys automatically; each commit produces a new image tag and therefore a new rollout.
8	Docker covers packaging, storage and networking on one host; Kubernetes adds scaling, self-healing, updates and stable service addressing.

Result

All Dockerfiles, Kubernetes manifests, the CI/CD workflow, commands, screenshots and observations for Lab 2 are recorded in one place and in the repository, so every experiment can be reproduced from this document. Task 9 is complete.